Úvod do React Native

Od webového sveta k mobilnému vývoju

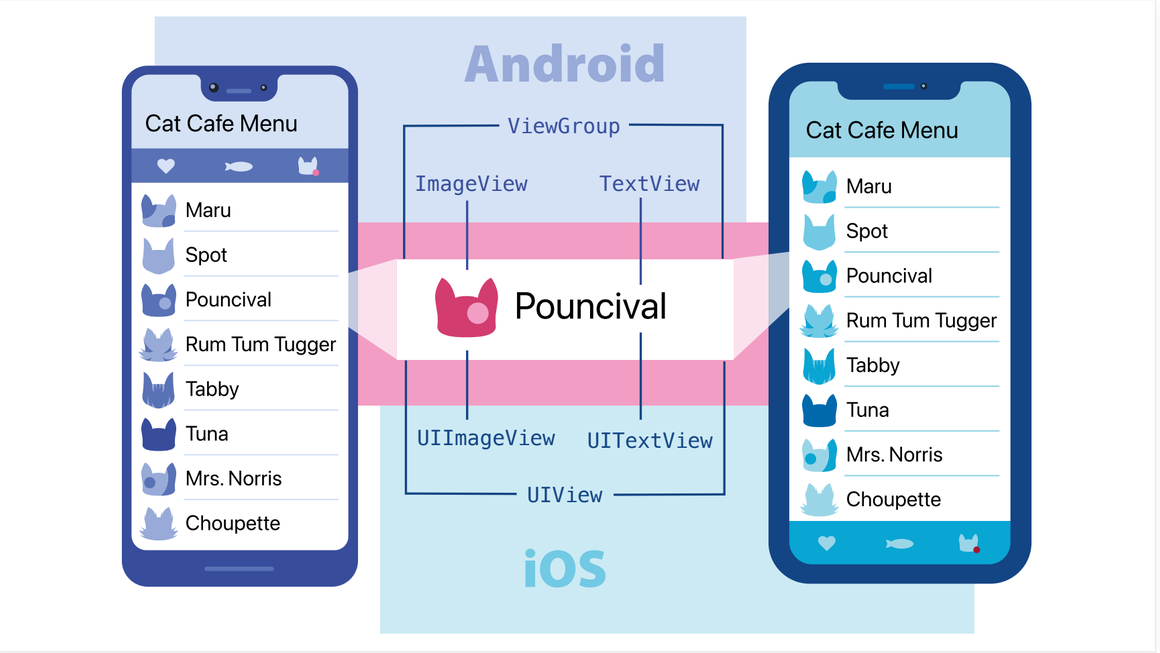
React Native Akadémia

React Native

- Open-source multiplatformový framework vyvinutý spoločnosťou Meta (Facebook).
- Umožňuje vývoj natívnych aplikácií pre iOS a Android pomocou JavaScriptu / TypeScriptu a **deklaratívneho princípu Reactu**.
- **Zásadný rozdiel oproti webu:** Nerenderuje aplikáciu vo vnútri webového prehliadača (WebView).
- Namiesto HTML elementov generuje **skutočné natívne UI komponenty** daného operačného systému:
  - `UIView` na platforme iOS
  - `android.view.View` na platforme Android

Čo je skutočným UI výstupom React Native aplikácie zobrazeným na displeji smartfónu?

1. HTML elementy renderované vo WebView
2. Natívne Android a iOS UI komponenty
3. Vlastné React Native komponenty vykresľované pixel-by-pixel
4. Statický obrázok vygenerovaný z React Native komponentov



Výstup projektu a distribúcia

Kompilačné balíčky

- Výstupom nie je webová stránka, ale natívna mobilná aplikácia.
- **iOS:** štandardný súbor `.ipa` (iOS App Store Package).
- **Android:**
  - `.aab` (Android App Bundle pre Google Play)
  - `.apk` (inštalačný balíček pre testovanie a vývoj)

Distribúcia do obchodov

- **Apple App Store:** upload cez App Store Connect / EAS Submit, schvaľovací proces App Review, beta cez TestFlight.
- **Google Play Store:** upload do Google Play Console, interné a produkčné tracky.
- Pre používateľa neexistuje rozdiel oproti aplikácii písanej v Swifte alebo Kotline.

Vznik React Native

DEKLARATÍVNE UI	KOMPONENTY	REKONCILIÁCIA

Čím bola zaujímavá prvá verejne dostupná open-source verzia React Native?

1. Renderovala aplikáciu vo WebView.
2. Podporovala len iOS platformu
3. Mala verziu 0.0.0
4. Bola publikovaná na medzinárodný deň programátorov.

React vs React Native

Vlastnosť	React (Web / ReactDOM)	React Native
Cieľové prostredie	Webový prehliadač (DOM)	Mobilný OS (iOS / Android natívne views)
Základné UI elementy	`<div>`, `<span>`, `<p>`, `<img>`, `<input>`	`<View>`, `<Text>`, `<Image>`, `<TextInput>`, `<ScrollView>`
Manipulácia s UI	Real DOM API (`document.createElement`)	Natívne volania OS (`UIView`, `android.view.View`)
Štýlovanie	CSS, Tailwind	JavaScript `StyleSheet` objekt + **Yoga** (Flexbox engine)
Interakcie	`onClick`, `onMouseEnter`, `onKeyDown`	`onPress`, `onLongPress`, Gesture Handler API
Špecifické API	Web API (`window`, `localStorage`, `fetch`)	Natívne API (`Camera`, `Geolocation`, `AsyncStorage`)
Balíčkovanie	Webpack, Vite, Rollup	**Metro Bundler** + **Hermes Engine**

Porovnanie syntaxe: Web vs Native

React (Web)

```tsx
import './UserProfile.css';

interface UserProfileProps {
  name: string;
  avatarUrl: string;
}

export function UserProfile({ name, avatarUrl }: UserProfileProps) {
  return (
    <div className="card" onClick={() => console.log('Pressed!')}>
      <img src={avatarUrl} alt={name} className="avatar" />
      <span className="name">{name}</span>
    </div>
  );
}
```

```css
.card {
  display: flex;
  flex-direction: row;
  align-items: center;
  padding: 12px;
  background-color: #ffffff;
  border-radius: 8px;
}

.avatar {
  width: 48px;
  height: 48px;
  border-radius: 24px;
  margin-right: 12px;
}

.name {
  font-size: 16px;
  font-weight: 600;
  color: #1a1a1a;
}
```

React Native

```tsx
import { Pressable, Image, Text, StyleSheet } from 'react-native';

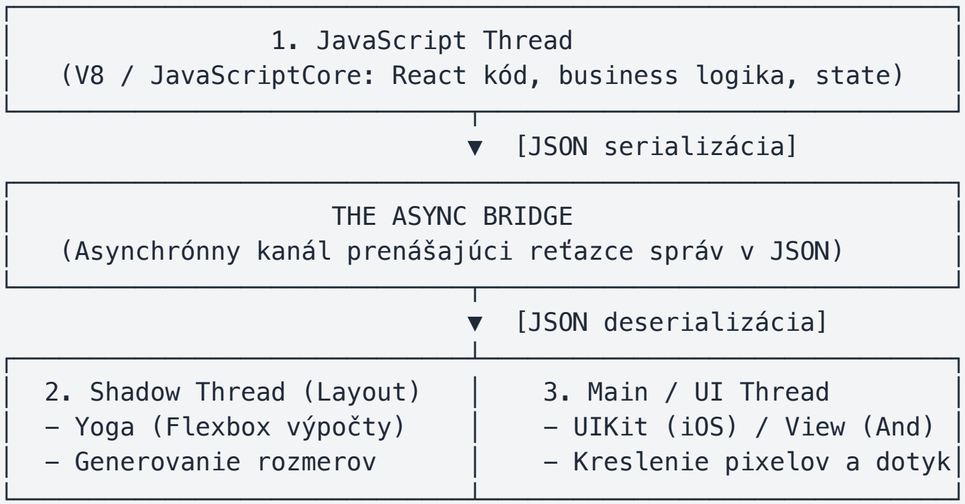
interface UserProfileProps {
  name: string;
  avatarUrl: string;
}

export function UserProfile({ name, avatarUrl }: UserProfileProps) {
  return (
    <Pressable style={styles.card} onPress={() => console.log('Pressed!')}>
      <Image source={{ uri: avatarUrl }} style={styles.avatar} />
      <Text style={styles.name}>{name}</Text>
    </Pressable>
  );
}

const styles = StyleSheet.create({
  card: {
    flexDirection: 'row',
    alignItems: 'center',
    padding: 12,
    backgroundColor: '#ffffff',
    borderRadius: 8,
  },
  avatar: {
    width: 48,
    height: 48,
    borderRadius: 24,
    marginRight: 12,
  },
  name: {
    fontSize: 16,
    fontWeight: '600',
    color: '#1a1a1a',
  },
});
```

React Native pod povrchom

Éra 1: Stará architektúra a "The Bridge"



Princíp fungovania (Message Passing)

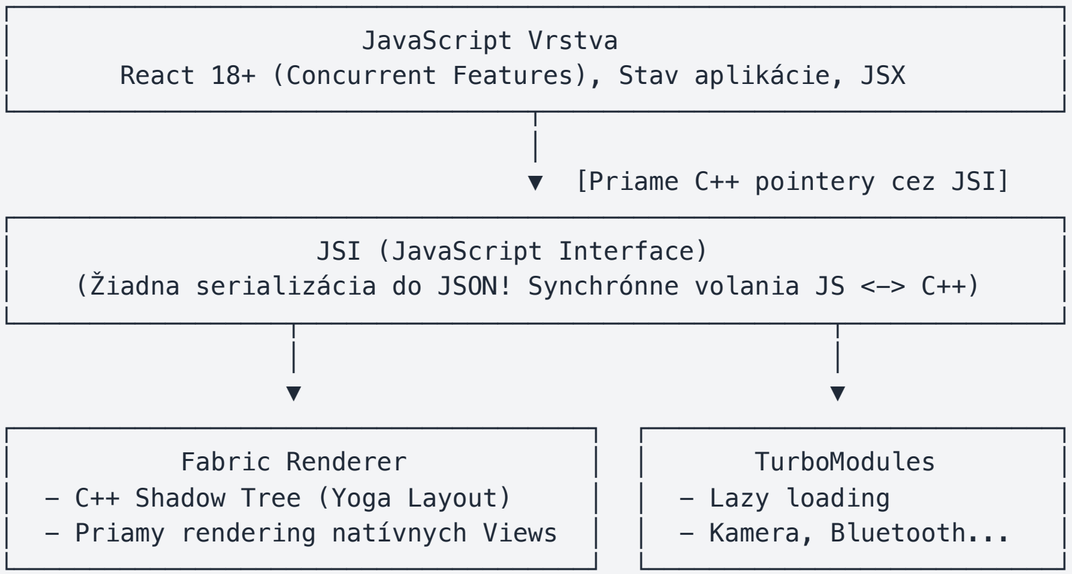
- JavaScript vlákno **nikdy nevolalo priamo** natívny kód (Objective-C / Swift / Java / Kotlin).
- Všetka komunikácia prebiehala na princípe správ:
  1. JavaScript zoserializoval volania a dáta do **JSON stringu**.
  2. JSON string sa odoslal cez **asynchrónny Bridge**.
  3. Na natívnej strane sa JSON rozparsoval (deserializoval).
  4. Natívny kód požiadavku vykonal a výsledok poslal rovnakým spôsobom späť.

Čo sa zvyklo zobrazovať na obrazovke pri starších React Native aplikáciách, ak ste začali rýchlo scrollovať dlhý zoznam?

1. Zamrznuté UI
2. Nič, celý zoznam zmizol
3. Duplicitné objekty zoznamu
4. Prázdne biele plochy

React Native pod povrchom

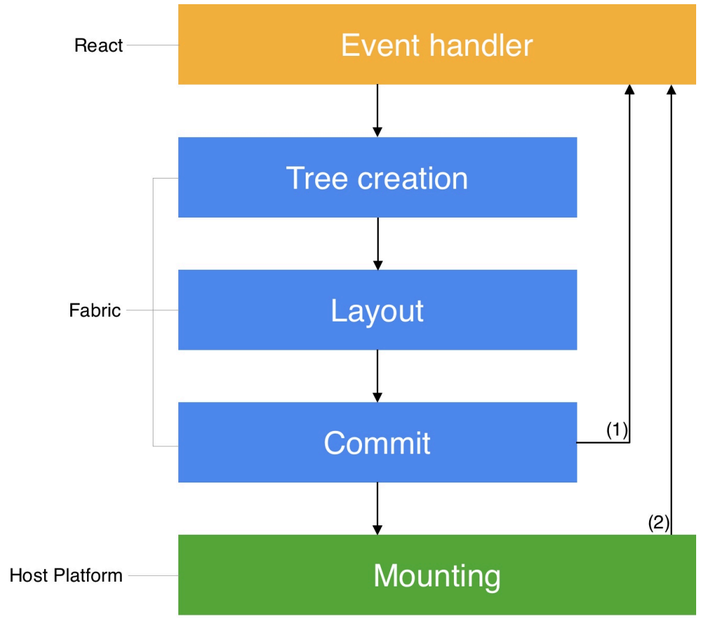
Éra 2: Nová architektúra (Modern RN)



- **JSI (JavaScript Interface):**
  - Žiadna serializácia do JSON reťazcov.
  - Možnosť **synchrónneho volania** natívnych metód priamo z JavaScriptu.
- **Hermes JavaScript Engine:**
  - Radikálne rýchlejší štart aplikácie (Time to Interactive).
  - Menšia spotreba pamäte RAM.
  - Menšia veľkosť inštalačného balíčka aplikácie.
- **Fabric Renderer:**
  - Logika generovania stromu prvkov (Shadow Tree) a výpočtu Flexbox layoutu (Yoga).
  - Rovnaká renderovacia logika na iOS aj Androide.
- **TurboModules:**
  - Lazy loading modulov.
  - Priame prepojenie s natívnym kódom cez JSI.

Render Pipeline (Fabric)

```
┌──────────────────────────────────────────────────────────────┐
│ 1. FÁZA: RENDER                                              │
│ React Element Tree (JS) ──> React Shadow Tree (C++)          │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 2. FÁZA: COMMIT                                              │
│ Výpočet layoutu (Yoga) ──> Povýšenie stromu (Tree Promotion) │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼
┌──────────────────────────────────────────────────────────────┐
│ 3. FÁZA: MOUNT                                               │
│ Diffing ──> View Flattening ──> Natívne zobrazenia (UI Thread)│
└──────────────────────────────────────────────────────────────┘
```



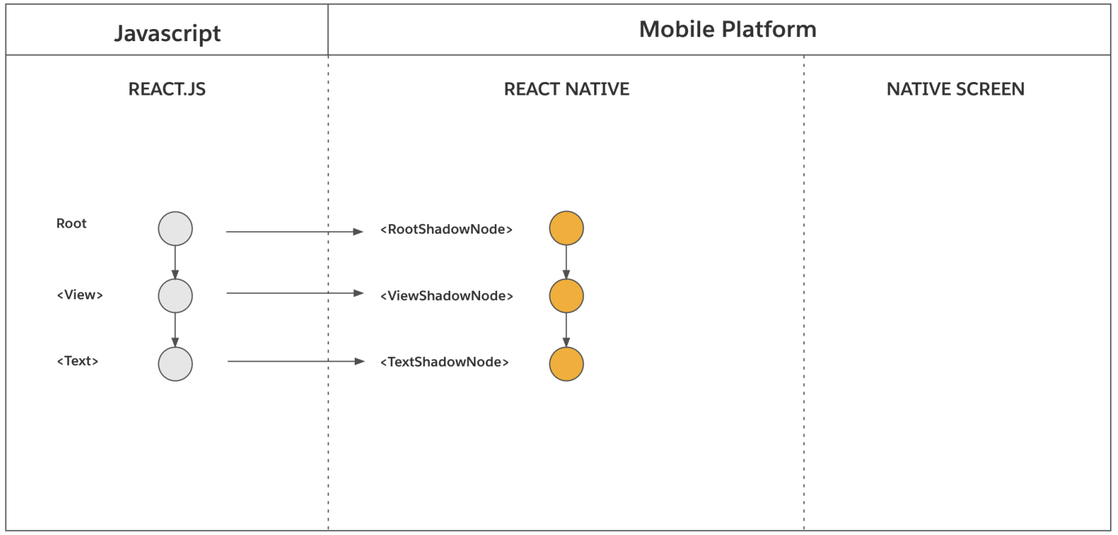
1. Events from core. For example: onLayout
2. Events from host platform. For example: onChange

Poznámka o vláknach: Tieto tri fázy môžu prebiehať na odlišných vláknach – Render a Commit môžu bežať asynchrónne na pozadí, zatiaľ čo Mount fáza prebieha vždy na hlavnom UI vlákne.

1. Fáza: Render

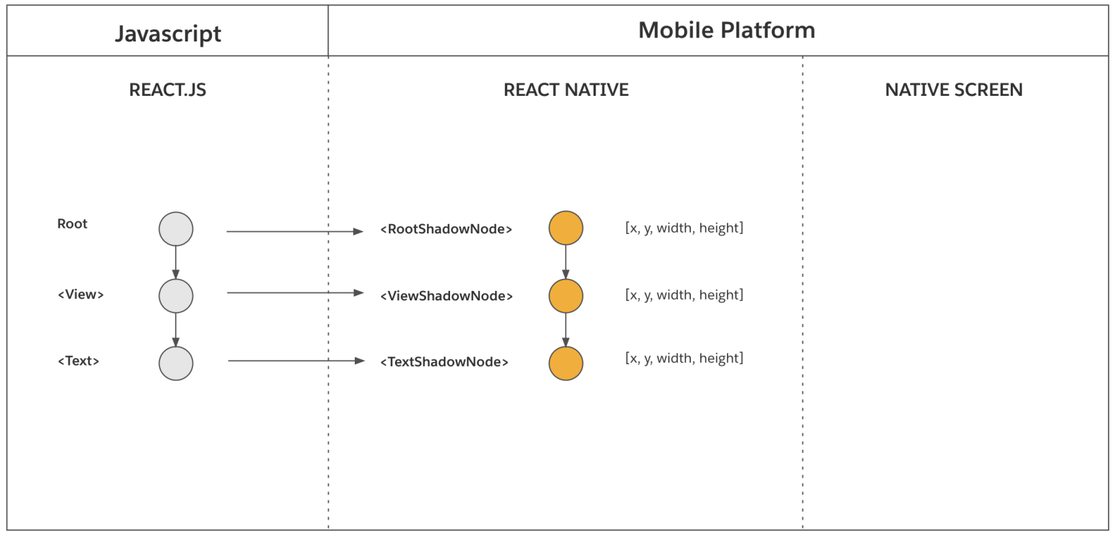
```
function MyComponent() {
  return (
    <View>
      <Text>Hello, World</Text>
    </View>
  );
}
```

- React postupne rekurzívne rozbaľuje vlastné komponenty.
- Vytvára sa React Element Tree v JS pamäti.
- Z tohto stromu Fabric renderer vytvorí v C++ paralelný React Shadow Tree zložený zo Shadow Nodes.
- Každý uzol tieňového stromu je striktne nemenný (immutable).



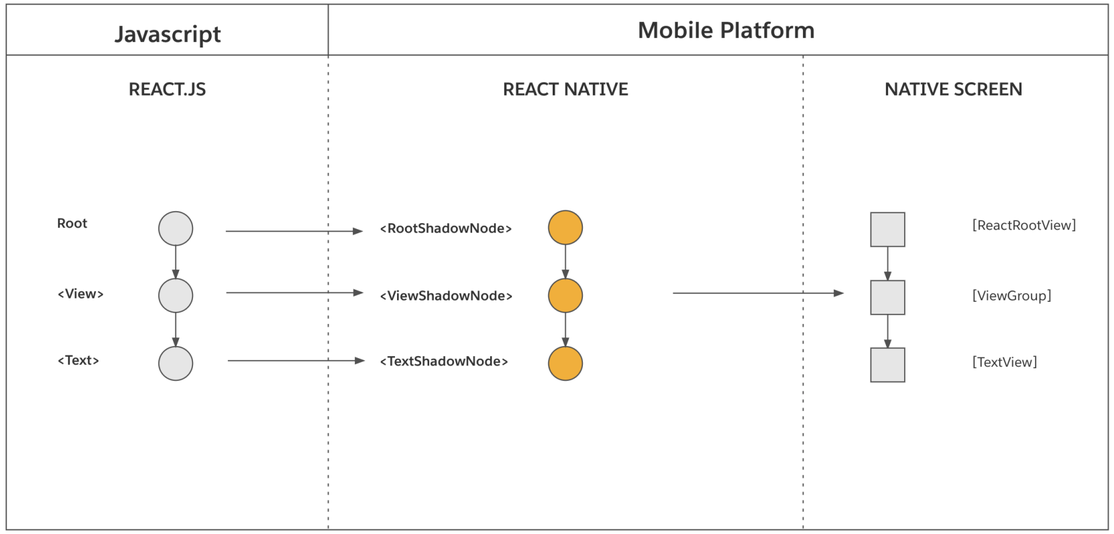
2. Fáza: Commit

- Prebieha na pozadí a zahŕňa dve hlavné operácie:
- **1. Výpočet layoutu (Layout Calculation):**
  - Vstavaný flexboxový engine **Yoga** prepočíta presné rozmery (šírku, výšku) a pozície (x, y) pre každý uzol tieňového stromu.
- **2. Povýšenie stromu (Tree Promotion):**
  - Nový C++ Shadow Tree sa stáva „ďalším stromom pripraveným na vykreslenie“ (Next Tree).
  - Signalizuje dokončenie fázy Commit a pripravenosť na zobrazenie.



3. Fáza: Mount

- Prebieha **výhradne na hlavnom UI vlákne**.
- **Diffing:** Porovná sa predchádzajúci tieňový strom s novým a zistia sa minimálne potrebné zmeny.
- **View Flattening:** Optimalizačný algoritmus, ktorý odstráni uzly slúžiace len na layout (bez pozadia, tieňov, orámovania).
  - Výsledkom je oveľa plytší strom natívnych pohľadov → obrovská úspora RAM!
- **Mutácia natívneho UI:** Vytvorenie a aktualizácia skutočných `UIView` / `android.view.View` prvkov na displeji.

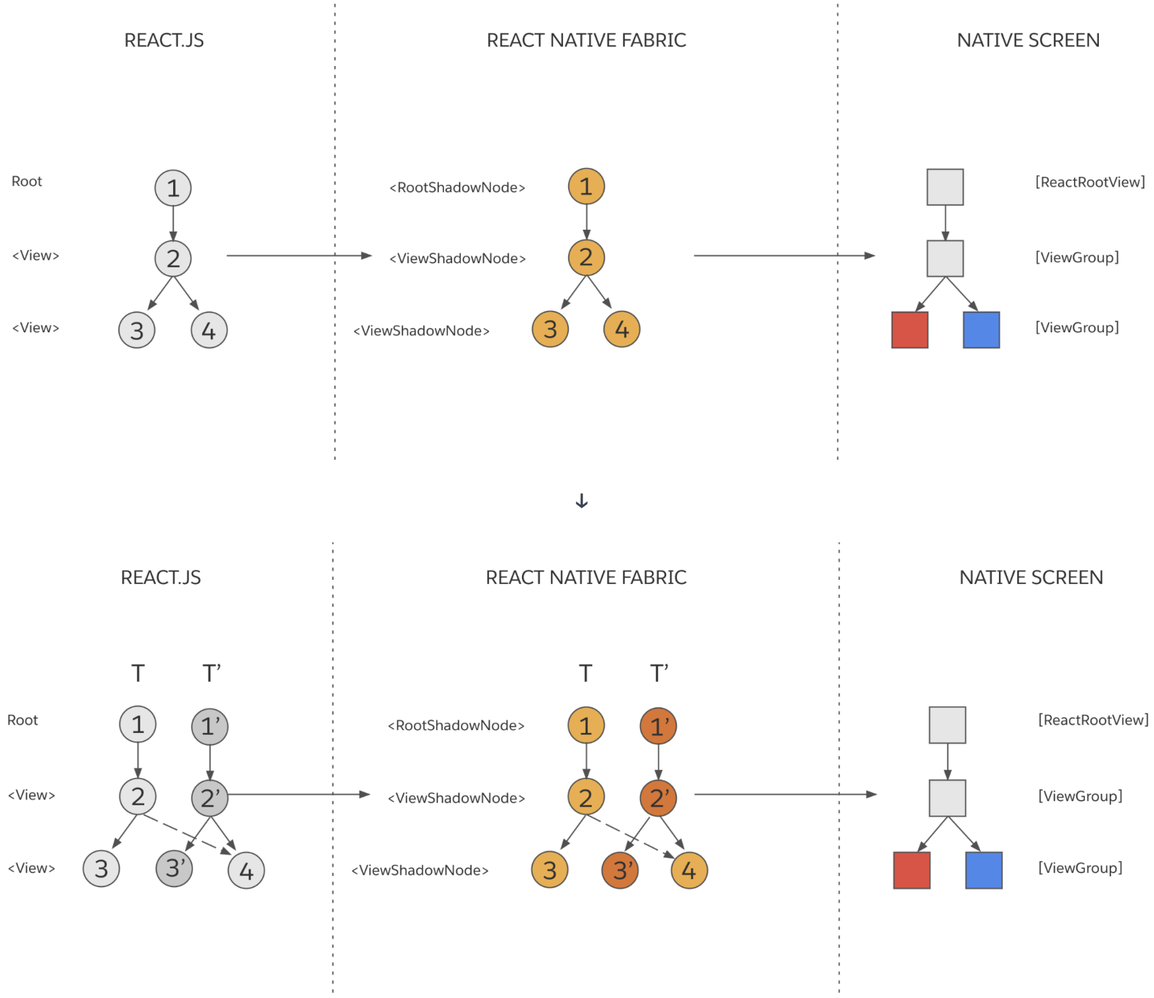


Aktualizácie stavu (React & React Native Renderer State Updates)

```
function MyComponent() {
  return (
    <View>
      <View
        style={{backgroundColor: 'red', height: 20, width: 20}}
      />
      <View
        style={{backgroundColor: 'blue', height: 20, width: 20}}
      />
    </View>
  );
}
```

React State Updates

- **Príklad:** Zmena farby pozadia prvého vnoreného `<View>` zmení z `'red'` na `'yellow'`.
- Prechádza sa kompletným cyklom: **Render → Commit → Mount**.
- Využíva sa štrukturálne zdieľanie v nemennom C++ strome.
- Identifikuje sa minimálny nutný rozsah zmien.



Výkon a snímková frekvencia

- Cieľom plynulého zážitku je dosiahnuť **60 snímok za sekundu (FPS)**.
- Rozpočet na jednu snímku: `1000 ms / 60 = 16,67 ms`.
- Ak spracovanie snímky trvá dlhšie ako 16,67 ms, snímka sa zahodí (**frame drop**).
- Používateľ to vníma ako sekanie, zadrhávanie alebo spomalenú odozvu rozhrania.
- React Native rozdeľuje výkon na dve nezávislé metriky:
  - **JS frame rate** (JavaScript thread)
  - **UI frame rate** (Main / UI thread)

JavaScript thread

- **Čo všetko tu beží:**
  - Biznis logika vašej aplikácie
  - Spracovanie touch eventov a API volaní
  - React render funkcie a výpočet nového stavu
- **Čo sa stane pri prepade JS FPS:**
  - Animácie riadené natívnym UI vláknom (napr. plynulý scroll) bežia ďalej hladko na 60 FPS
  - Aplikácia však nereaguje na kliknutia (tlačidlá nereagujú, navigácia mešká).

Main (UI) thread

- **Čo tu prebieha:**
  - Fyzické kreslenie natívnych pohľadov na displej
  - Gestá a scrollovanie v operačnom systéme
  - Animácie nakonfigurované s príznakom `useNativeDriver: true`
- **Prepad UI frame rate:**
  - Okamžite viditeľný ako sekanie animácií, trhanie zoznamu pri scrollovaní.
  - Najhorší dopad na vnímaný používateľský zážitok (UX).



TIME FOR

localhost.company

HYDRATION BREAK

React Native & Expo

„Bare" React Native (Community CLI)

- Pôvodný spôsob vývoja cez príkaz `npx @react-native-community/cli init`.
- Projekt od prvého dňa obsahuje fyzické priečinky `ios/` (Xcode) a `android/` (Android Studio).
- **Vyžaduje plnú znalosť natívnych nástrojov:** CocoaPods, Gradle, manuálne prepájanie knižníc, riešenie konfliktov závislostí.
- **Pre koho je určený:**
  - Vývojári integrujúci React Native do existujúcej obrovskej natívnej aplikácie (Brownfield vývoj).
  - Tímy s dedikovanými iOS a Android inžiniermi s vlastnou CI/CD infraštruktúrou.

Expo Framework (Moderný štandard & CNG)

- Plnohodnotný produkčný framework pre React Native (ekvivalent Next.js pre webový React).
- Oficiálne odporúčaný tímom React Native pre všetky nové projekty.
- **CNG (Continuous Native Generation):**
  - Priečinky `ios/` a `android/` sa nespravujú ručne v Gite.
  - Generujú sa automaticky na požiadanie z jedného súboru `app.json` cez **Expo Prebuild**.
- **Koniec „Ejectovania":** Expo už dávno nie je uzatvorená platforma – vďaka Config Plugins podporuje ľubovoľný vlastný natívny kód v Swifte/Kotline!

Aký štandard nám odporúča React RN tím použiť pre vývoj mobilných aplikácií?

1. Natívny iOS a Android
2. Web + PWA
3. Bare React Native
4. React Native + Expo

Zhrnutie: Bare RN vs. Expo

„Bare" React Native

- `@react-native-community/cli`
- `ios/` a `android/` priečinky pevne v Gite
- Konfigurácia roztrúsená v Xcode, Gradle a plist súboroch.
- Manuálny upgrade verzií cez git diff natívnych súborov.
- Prístup k vlastnému natívnemu kódu.
- Vyžaduje Xcode a Android Studio a lokálny Mac pre iOS buildy od prvého dňa.
- Vhodný najmä pre Brownfield integrácie.

Expo Framework

- `create-expo-app`
- `ios/` a `android/` priečinky generované automaticky podľa potreby
- Jediný konfiguračný súbor `app.json`.
- Automatizovaný upgrade verzií pregenerovaním kódu.
- Prístup k vlastnému natívnemu kódu cez Development Builds.
- Možnosť buildovať v cloude (EAS Build) aj bez vlastného Macu.
- Oficiálne odporúčaný framework pre všetky nové projekty.

Čo všetko Expo poskytuje?

- **Expo SDK:** Obrovská knižnica vyladených multiplatformových balíčkov (kamera, biometria, audio, notifikácie, súborový systém).
- **Expo Router:** Súborový navigačný systém (File-based routing) známy z moderného webu (Next.js), plne typovaný v TypeScript.
- **EAS (Expo Application Services):** Profesionálna cloudová platforma na zostavovanie balíčkov (EAS Build), over-the-air updaty (EAS Update) a odosielanie do obchodov (EAS Submit).
- **Expo Go:** Mobilný pieskovisko klient pre okamžité testovanie aplikácie na reálnom telefóne bez nutnosti kompilácie.

Expo Go vs. Dev Builds vs. Prebuild

- **Expo Go (Začiatok):**
  - Nainštalujete z App Store / Google Play, naskenujete QR kód a kód beží za 2 sekundy.
  - Obsahuje predinštalované bežné Expo knižnice; nepodporuje vlastný C++/natívny kód tretích strán.
- **Development Builds (Pokročilý vývoj):**
  - Vlastná natívna verzia Expo Go skompilovaná priamo pre váš projekt.
  - Umožňuje integrovať akýkoľvek vlastný natívny modul (Stripe, Bluetooth, vlastný Swift/Kotlin).
- **Expo Prebuild (Generátor):**
  - Nástroj, ktorý vezme `app.json` a Config Plugins a vygeneruje čisté `ios/` a `android/` projekty.

Vytvorenie prvého projektu

Požiadavky (Prerequisites):

- **Node.js (LTS verzia):** Odporúčaná verzia 22.x alebo 24.x (stiahnete z nodejs.org).
- **Git:** Pre správu verzií zdrojového kódu.
- **Terminál / Príkazový riadok:** Bash, Zsh, alebo PowerShell.
- **Editor kódu:** Odporúčaný **Visual Studio Code** s rozšíreniami pre React Native a Expo.
- **Mobilný telefón s aplikáciou Expo Go:**
  - iOS: stiahnuť z Apple App Store
  - Android: stiahnuť z Google Play

Predvolená moderná šablóna (Expo Router + TypeScript):

```
npx create-expo-app@latest my-first-rn-app
```

Alternatíva: Úplne čistý TypeScript projekt:

```
npx create-expo-app@latest my-first-rn-app --template blank-typescript
```

Prechod do priečinka a spustenie:

```
cd my-first-rn-app
npx expo start
```

Popis štruktúry projektu

```
my-first-rn-app/
├── assets/                  # Statické assety v roote (ikona, splash screen)
│   └── images/
├── scripts/                 # Pomocné skripty projektu (reset-project)
├── src/                     # Zdrojový kód aplikácie
│   ├── app/                 # Expo Router - navigačná štruktúra (file-based)
│   │   ├── (tabs)/          # Skupina obrazoviek so spodnou navigáciou (Tab Bar)
│   │   │   ├── index.tsx    # Úvodná obrazovka (Home Tab)
│   │   │   └── explore.tsx  # Sekundárna obrazovka (Explore Tab)
│   │   ├── +not-found.tsx   # Fallback obrazovka pre neexistujúcu trasu (404)
│   │   └── _layout.tsx      # Koreňový layout celej aplikácie
│   ├── components/          # Vlastné znovupoužiteľné React komponenty
│   ├── constants/           # Konštanty, farby témy
│   └── hooks/               # Vlastné React hooky
├── app.json                 # Hlavná konfigurácia Expo a natívnych vlastností
├── package.json             # NPM závislosti a spúšťacie skripty
└── tsconfig.json            # Konfigurácia TypeScriptu
```

Súbory v koreňovom priečinku (Root)

- `app.json:` Mozog celého projektu. Definuje názov, ikonu, splash screen a natívne povolenia pre iOS aj Android.
- `package.json:` Zoznam závislostí a spúšťacích príkazov.
- `tsconfig.json:` Konfigurácia TypeScriptu.
- `assets/:` Statické vizuálne súbory (ikony, splash screen).
- `scripts/:` Pomocné servisné skripty (napr. `reset-project`).

Kód v priečinku src/ (Expo Router)

- `src/app/`: Jadro navigácie cez **Expo Router**.
  - Každý súbor v tomto priečinku automaticky reprezentuje obrazovku v aplikácii (file-based routing).
  - `_layout.tsx`: Zdieľaný obal (layout), definuje navigačný stack, motív alebo globálne kontexty.
  - `(tabs)/`: Zoskupenie obrazoviek so spodnou záložkovou lištou (Tab Bar).
  - `+not-found.tsx`: Obrazovka 404 pri zadaní neplatnej cesty.
- `src/components/`: Vlastné znovupoužiteľné UI komponenty.
- `src/constants/`: Centrálne hodnoty (farby, témy, rozmery).
- `src/hooks/`: Zdieľané React hooky (napr. sledovanie dark mode).

app.json

```
{
  "expo": {
    "name": "my-first-rn-app",
    "slug": "my-first-rn-app",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/images/icon.png",
    "scheme": "myapp",
    "userInterfaceStyle": "automatic",
    "splash": {
      "image": "./assets/images/splash-icon.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "ios": {
      "supportsTablet": true
    },
    "android": {
      "adaptiveIcon": {
        "foregroundImage": "./assets/images/adaptive-icon.png",
        "backgroundColor": "#ffffff"
      }
```

Spustenie projektu

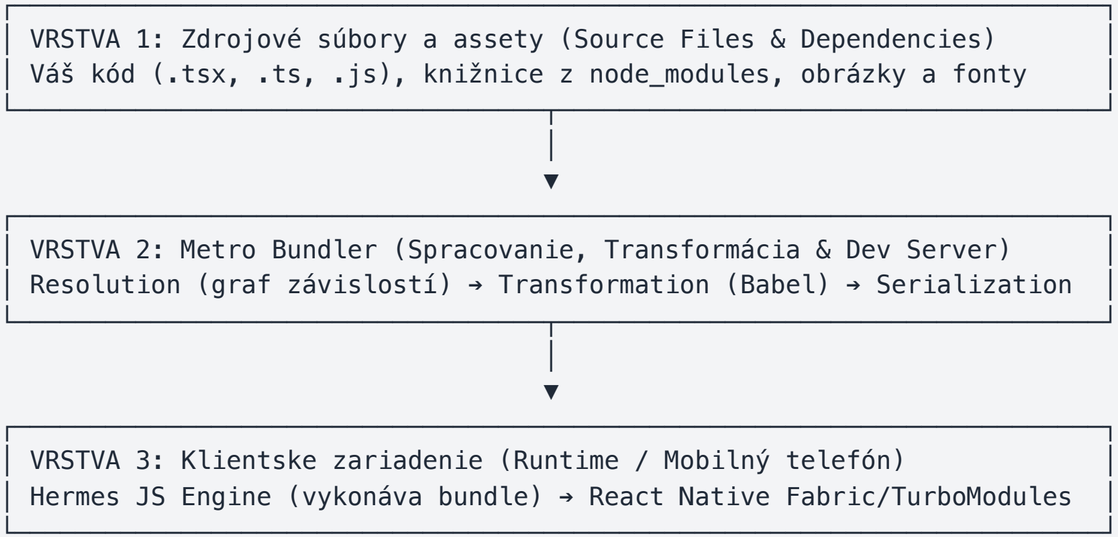
```
npx expo start
```

Metro Bundler

- Dedikovaný a optimalizovaný JavaScriptový bundler vyvinutý pre React Native (ekvivalent Webpacku / Vite).
- Zbalí stovky súborov, `node_modules` a assety do jediného balíka pre JS engine (Hermes).

3 fázy spracovania

1. Resolution (Riešenie závislostí)
2. Transformation (Transformácia kódu)
3. Serialization (Serializácia a zostavenie balíka)



Ako Metro funguje pod povrchom

1. Resolution (Riešenie závislostí):

- Začína od entry pointu (`src/app/_layout.tsx`).
- Prechádza `import` / `require()` a rekurzívne stavia kompletný **graf závislostí**.
- Rieši platformové prípony (na iOS uprednostní `Button.ios.tsx` pred `Button.tsx`).

2. Transformation (Transformácia kódu):

- Paralelná transpilácia cez Babel / SWC do formátu pre Hermes engine.
- Agresívne viacúrovňové cachovanie → prekladá sa len upravený súbor!

3. Serialization (Serializácia a spojenie):

- Spojí moduly do finálneho JavaScript balíka
- Vygeneruje Source Maps

Metro: Vývojový režim vs. Produkcia

Vývojový režim (Development)

- Metro beží ako lokálny **HTTP a WebSocket server** na porte `8081`.
- Aplikácia si sťahuje bundle cez Wi-Fi alebo USB.
- **Fast Refresh (HMR):** Pri uložení v editore pošle cez WebSocket len zmenený modul.
- Zmena sa prejaví za zlomok sekundy **bez straty stavu (state)**.

Produkčný build (Release)

- Metro ako bežiaci server **vôbec nevystupuje**.
- Vopred sa vygeneruje statický bundle.
- Bundle sa predkompiluje do optimalizovaného **Hermes Bytecode** (`.hbc`).
- Pribalí sa priamo do inštalačného balíčka (`.apk` / `.ipa`) → okamžitý offline štart!

Možnosti spustenia aplikácie

1. **Fyzický telefón cez Expo Go (najrýchlejšie):**
   - **iOS:** Otvorte aplikáciu Fotoaparát, naskenujte QR kód z terminálu a otvorte notifikáciu.
   - **Android:** V aplikácii Expo Go zvoľte „Scan QR code".
   - Podmienka: Počítač aj telefón musia byť na **rovnakej Wi-Fi sieti**.
2. **iOS Simulátor:** V termináli stlačte klávesu `i` (vyžaduje macOS a Xcode).
3. **Android Emulátor:** V termináli stlačte klávesu `a` (vyžaduje Android Studio).
4. **Webový prehliadač:** V termináli stlačte klávesu `w` (otvorí sa cez `react-native-web`).

Vývojársky cyklus (DX)

Rýchly cyklus spätnej väzby:

- **Fast Refresh:** Uložíte ľubovoľný `.tsx` súbor a zmeny sa okamžite prejavia na displeji **bez straty stavu komponentov**.
- **Developer Menu:**
  - V simulátore: `Cmd + D` (iOS) alebo `Cmd + M` (Android).
  - Na fyzickom zariadení: **zatraste telefónom (shake gesture)** alebo klepnite tromi prstami.
  - Možnosti: manuálny reload, Element Inspector, otvorenie React DevTools.

Zhrnutie lekcie

1. **React Native** spája vývojársky komfort a deklaratívnu silu Reactu s výkonom a používateľským zážitkom skutočných natívnych aplikácií.
2. Namiesto HTML elementov manipuluje priamo **natívne komponenty systému** (`UIView` / `android.view.View`).
3. **Nová architektúra** odstránila asynchrónny Bridge a priniesla priame prepojenie **JSI**, renderer **Fabric**, engine **Hermes** a **TurboModules**.
4. **Render Pipeline** pozostáva z 3 fáz: **Render** (nemenný C++ tieňový strom), **Commit** (Yoga layout výpočty) a **Mount** (diffing, view flattening, natívne zobrazenia na UI vlákne).

5. **Snímková frekvencia (Frames):** Cieľom je 60 FPS (16,67 ms limit). Meria sa **JS frame rate** (biznis logika, touch) a **UI frame rate** (kreslenie pixelov).
6. **Expo** je odporúčaným štandardom: odstraňuje manuálnu správu v Xcode/Android Studiu a prináša Continuous Native Generation (Prebuild).
7. Štart cez `create-expo-app` a `expo start` poskytuje okamžitý Fast Refresh na telefóne aj v simulátore.

Odporúčané zdroje:

- reactnative.dev – Oficiálna dokumentácia
- docs.expo.dev – Expo dokumentácia
- Render Pipeline Guide
- Hermes Engine Architecture
- Yoga Layout Engine